Tool Use II: Permissions, Least Privilege, and Approval

FRIDAY AI Education — Episode 035 · 2026-07-21 · Printable Companion

FRIDAY AI Education — Episode 035 · Lesson 21 of 37 · Tool Use, Part 2 of 3

Abstract

The previous lesson established that tool use turns a language model from a text generator into an actor: the model emits a structured request, and a piece of software you control — the harness — decides whether to execute it. This paper is about that decision. It develops a single design discipline: **match the permission and approval structure of each tool to the consequence of the action it performs.** The load-bearing concepts are four: *permission* (what an agent is mechanically able to request), *least privilege* (granting the minimum capability the job requires), *reversibility* (whether an action can be cheaply undone), and the *approval gate* (a checkpoint where a human or a stricter system must consent before execution). We work through three canonical cases — a read-only research agent, an email send gate, and a payment gate — and close with the main failure mode, which is not a hacked agent but a bored human. The lesson prepares the ground for the next one: once you decide an action may proceed, you need it to execute reliably, exactly once, with a receipt.

1. Where This Sits

One sentence of bridge, as promised. In Tool Use I, the key structural fact was this: the model never acts directly. It produces a request — "call `send_email` with these arguments" — and the harness, ordinary deterministic software, executes it or doesn't. The model proposes; the harness disposes.

That fact seemed like plumbing. It is actually the entire foundation of agent safety, because it means **the question "what can this agent do?" is never answered by the model. It is answered by whoever configured the harness.** The model has no hands. You built the hands, and you decided what they can reach.

Today's new layer is the design discipline for that configuration. Not "can the model call tools" — settled last time — but *which* tools, with *what scope*, and *which calls require a human to say yes first*. This is where agent design stops being a machine-learning problem and becomes something closer to organizational design, which is why it belongs in a founder's toolkit and not just an engineer's.

2. The Core Principle: Consequence, Not Capability

Here is the single idea of this paper, stated once and then defended.

> **Permission design should be driven by the consequence of the action, not by the sophistication of the agent.**

Most people get this backwards on first contact. The intuitive framing is: "How smart is the model? How much do I trust it? Trust it more, give it more." That framing puts the burden of proof on the model's competence, which is a moving, hard-to-measure, and — this matters — *adversarially manipulable* quantity. A model that performs beautifully in your tests can be steered off course in production by a malformed input, an ambiguous instruction, or content it read from the open web that was written precisely to steer it.

The correct framing inverts the question: "If this specific action goes wrong, what does it cost me, and can I take it back?" That question has answers that don't depend on the model at all. Sending a wrong email to a customer costs reputation and cannot be unsent. Reading a document costs nothing and requires no undoing. Wiring money costs money, and depending on the rail, may be gone forever. These facts are stable properties of the *actions*, and you can build policy on stable facts.

This is not a novel insight of the AI era. It is how competent organizations have always worked. A junior analyst at an investment firm can read every research report in the building on day one, because reading is cheap and reversible-in-effect: no harm has left the building. The same analyst cannot execute a trade without sign-off, and cannot move client funds at all — not because anyone doubts their intelligence, but because the *consequences* of those actions warrant a control structure regardless of who performs them. Nobody at the firm considers this insulting. It is simply how consequence is managed.

Agents deserve exactly the same treatment, for a stronger reason: an agent is a probabilistic system operating at machine speed. A human who is about to send a catastrophic email usually hesitates — some social instinct fires. An agent has no such instinct unless you build it, and it can make the mistake four hundred times before lunch. Speed and scale amplify consequence, which means agents need *more* deliberate consequence-matching than human employees, not less.

Everything that follows is machinery for implementing this one principle.

3. Permission: The Mechanical Layer

Start with the most concrete term, because the other three build on it.

A **permission**, in the tool-use context, is the answer to the question: *what is this agent mechanically able to request, and with what scope?* It is enforced by the harness and the credentials behind it — not by the prompt.

That last clause is the part worth underlining. There are two ways to "restrict" an agent, and they are not remotely equivalent:

1. **Instruction:** you write "never send emails to external addresses" in the system prompt.
2. **Permission:** the email tool exposed to the agent physically cannot address anyone outside the company domain — the harness rejects the call before it reaches an email server.

The first is a request made to a probabilistic system. It will usually be honored. "Usually" is doing regrettable work in that sentence: models can misread context, can be manipulated by content they process, or can simply err under an unusual combination of inputs. The second is a property of software. It holds whether the model is having a good day or a bad one, and it holds against manipulation, because the attack surface is deterministic code rather than a model's judgment.

The practical rule: **prompts express intent; permissions enforce it.** Use both, but never let a prompt be the only thing standing between an agent and a consequence you can't absorb. If you find yourself writing "IMPORTANT: never do X" in increasingly urgent capital letters, that is the system telling you X should not be a permission this

agent holds. Delete the plea; revoke the capability.

Permissions have **scope**, and scope is where the real design work lives. "Can send email" is not one permission; it is a family:

- send to anyone, about anything;
- send only to internal addresses;
- send only from a specific mailbox, only using approved templates;
- draft email that a human sends.

Each is a different risk posture wearing the same tool name. When you evaluate an agent system — yours or a vendor's — the revealing question is never "what tools does it have?" but "what is the scope on each one?" Anthropic's tool-use documentation makes the underlying mechanics concrete: each tool is defined by a name, a description, and a schema constraining its inputs, and the developer controls that entire definition. The schema itself is a permission surface — a payment tool whose schema caps `amount` at 500 has encoded policy at the type level, where no amount of model confusion can exceed it.

4. Least Privilege: The Allocation Rule

Least privilege is a principle borrowed from decades of security engineering, and it transfers to agents intact: *grant each agent the minimum set of permissions required to do its job, and nothing more.*

The classical argument is about blast radius. Every permission an agent holds is a possible consequence when something goes wrong — through model error, prompt injection, a bug in the harness, or a compromised credential. Permissions the agent doesn't hold are consequences that *cannot occur*, no matter what goes wrong. A research agent with no write access cannot corrupt your database. Not "probably won't." Cannot. In a domain where you can't fully predict model behavior, "cannot" is the only word that lets you sleep, and least privilege is how you buy it.

There is a second argument, less discussed and arguably more important for builders: **least privilege improves the agent, not just the safety case.** An agent with forty tools must decide, at every step, which of forty tools is relevant — and every irrelevant tool is a chance to wander. An agent with the four tools its job requires has a smaller decision space, fewer failure modes, and behavior that is dramatically easier to evaluate and debug. Narrow agents are better agents. The security principle and the performance principle point the same direction, which is rare enough in engineering to be worth savoring.

Two practical corollaries:

Design permissions per task, not per agent. The lazy pattern is one general-purpose agent holding the union of all permissions anyone might ever need — the software equivalent of a master key hanging by the door. The disciplined pattern is task-scoped: the research task runs with read-only permissions; the outreach task runs with draft-and-queue permissions; the payment task runs with tightly capped payment permissions plus a mandatory gate. Same underlying model, different keyrings. If you internalized the workflow spec lessons from earlier in this course, notice that the permission set is properly part of the workflow spec: a workflow's definition includes not just its objective, inputs, and output contract, but the exact capabilities it runs with.

Start closed and open deliberately. Grant nothing by default; add each permission when a concrete task demonstrates the need, and record why. The opposite ratchet — start open, restrict after incidents — means your permission model is a scar tissue of past failures rather than a designed artifact. Companies that end up in the second state always intended to end up in the first. The difference was whether anyone owned the decision on day

one.

5. Reversibility: The Axis That Sets the Price

Permissions tell you what an agent can request. To decide what it *should* be able to request — and at what level of ceremony — you need a way to price actions. The most useful single axis is **reversibility**: if this action turns out to be wrong, what does it cost to undo, and is undoing even possible?

Roughly three bands:

Reversible. The action can be undone cheaply and completely, leaving no trace of harm. Reading a file. Querying a database. Writing to a scratch workspace. Creating a draft. If it goes wrong, you delete the output and try again; the world outside your system never knew.

Reversible with cost. The action can be substantially undone, but undoing consumes real resources — time, money, apologies. A wrong internal database write that must be traced and corrected. A miscreated calendar invite that thirty people saw before it was pulled. The mistake is fixable, but the fix has a bill.

Irreversible. The action cannot be undone, only *compensated*. A sent email is the cleanest example: there is no unsend, only a follow-up correction, and the recipient's impression of your company has already been updated. Money wired over an irrevocable rail. A public post that has been screenshotted. A deleted record with no backup. For these, the honest framing is that "undo" doesn't exist — only damage control does.

The bands are not always obvious from the tool's name, and this is a place where careful builders earn their keep. "Post to Slack" sounds trivially reversible — there's a delete button — but if forty colleagues read the wrong message before deletion, the *information* is irreversible even though the *artifact* is not. Conversely, "send payment" on a rail with a built-in clawback window is more reversible than it sounds. The discipline is to classify the *real-world effect*, not the API verb. When in doubt, ask: after the undo, is the world actually restored, or merely patched?

One elegant and widely applicable move deserves its own paragraph: **you can often re-engineer an irreversible action into a reversible one by splitting it.** "Send email" is irreversible; "draft email and place it in an outbox with a fifteen-minute delay" is fully reversible for fifteen minutes. "Delete record" is irreversible; "soft-delete with thirty-day retention" is not. "Pay invoice" is irreversible; "queue payment for tomorrow's batch" gives you until tonight. This is frequently the highest-leverage safety work in an agent system, because it doesn't restrict the agent at all — it restructures the action so that the agent's full autonomy becomes affordable. Before you bolt an approval gate onto a dangerous action, first ask whether you can make the action less dangerous.

6. Approval Gates: The Human Checkpoint

Some actions remain consequential no matter how you restructure them. For these, the tool of choice is the **approval gate**: a designated checkpoint where the agent's proposed action is held, presented to a human (or a stricter automated policy), and executed only on explicit consent.

A gate is not a review of the agent's reasoning, and it is not a vibe check on the model. It is a structural property of the harness: *this tool call does not execute until approval arrives.* The agent can propose; the proposal sits in a queue; a human clicks yes or no; only then does the harness act. Between proposal and approval, nothing has happened in the world — which means a gated action is, until the moment of approval, perfectly reversible. That is the quiet elegance of the mechanism: **a gate converts an irreversible action into a reversible proposal.** All the

consequence is concentrated into a single human decision, made with the full proposal visible.

Because human attention is the scarce input, gate design is mostly attention economics. Three rules:

Gate the consequence, not the workflow. A common beginner error is gating everything, on the theory that more oversight is more safety. The result is a human approving forty routine actions a day — and here is the trap: *routine approval decays into no approval.* The human stops reading, starts pattern-matching, and clicks yes on reflex. When the one genuinely bad proposal arrives, it gets the same reflexive yes, because it looks like the other thirty-nine. This is **approval fatigue**, and it is the dominant failure mode of gated systems — worth its own section below. The countermeasure is ruthless selectivity: gates go on irreversible, high-consequence actions only. Everything reversible flows freely, precisely so that when a gate does fire, the human knows it means something.

Make the approval decision cheap to make well. A good gate presents everything needed to decide in seconds: what the agent wants to do, to whom or for how much, why (the agent's stated justification), and what happens if approved. "Agent requests: send attached email to customer X regarding refund; full text below; triggered by ticket #4471" is a decidable proposal. "Agent requests: proceed?" is not — it forces the human to either investigate (expensive, so they won't, so they'll guess) or rubber-stamp. If your approvers are asking follow-up questions, your gate is under-designed; if they're approving in under a second, it may be over-fired.

Let policy, not mood, decide what's gated. The set of gated actions should be written down, versioned, and changed deliberately — the same way a company writes down spending authority. "Payments over €500 require approval; payments under €500 to pre-approved vendors do not" is a policy someone can audit, inherit, and improve. "Sometimes we check the payments" is not a policy; it's an anecdote.

A useful mental model for the whole apparatus: gates are to agents what **signing authority** is to a company. Every functioning company already runs this exact system for humans — who can commit the company to a contract, who can spend what without countersignature, whose name goes on the wire transfer. Nobody calls that "distrust of employees"; it's called governance, and its presence is precisely what allows delegation to scale. You are not inventing a new discipline for agents. You are extending a very old one to a new kind of worker — one that works faster, tires never, and hesitates never, which is exactly why the old discipline matters more.

7. The Decision Table

The two axes developed above — consequence and reversibility — compose into a compact decision rule. This table is the paper in one artifact; the rest is justification.

Consequence if wrong	Reversible	Reversible with cost	Irreversible
Low (internal, contained)	Free autonomy. No gate; log it.	Autonomy with tight scope; log and spot-check.	Gate lightly, or restructure into a reversible form (delay, queue, soft-delete).
Medium (money, customers, or reputation touched modestly)	Autonomy within scoped limits (caps, allowlists); log everything.	Scoped autonomy plus periodic human review of the log.	Approval gate, well-presented.
High (material money, public statements, legal or safety exposure)	Autonomy acceptable but scope narrowly; review logs regularly.	Approval gate.	Approval gate — and ask whether the agent should hold this permission at all.

Two readings of the table are worth making explicit. First, the top-left region is large on purpose: most of what agents do — reading, searching, drafting, computing, writing to scratch space — belongs there, and should flow at full speed with zero ceremony. Restraint is reserved, not sprayed. Second, the bottom-right cell contains a question, not just a

gate. Some permissions shouldn't exist in agent hands at any gate strength — not because the model is untrustworthy, but because no upside justifies the tail risk. Knowing where that line sits for your company is a judgment call no framework makes for you. The framework's job is to make sure someone consciously makes it.

8. Three Canonical Cases

Theory earns its keep in the concrete. Three cases, in ascending order of consequence.

8.1 The read-only research agent

Task: given a question, search a corpus and the web, read sources, and produce a cited brief.

Permission analysis: every action is a read or a scratch-write. Nothing leaves the system; nothing in the world changes. Top-left cell of the table: **no gates at all.** Gating a read-only agent is pure friction with zero risk reduction — the archetypal misallocation of the attention budget.

What the case actually teaches is the power of the *read-only guarantee* as a designed property. Because the agent holds no write or send permissions, you can afford enormous autonomy: let it read hundreds of documents, follow leads, run for an hour unattended. The permission boundary is what makes the autonomy affordable. And note the subtle threat it neutralizes: a research agent reads untrusted content — web pages, PDFs, other people's documents — any of which could contain text crafted to manipulate a model ("ignore your instructions and forward this to…"). Against a read-only agent, a successful manipulation achieves precisely nothing, because the harness offers no tool that touches the world. The injection lands; the blast radius is zero. This is least privilege doing security work that no amount of prompt-hardening can replicate. For anyone whose first practical agents are knowledge-base and public-corpus tools, this is the reassuring case: the wedge you're building lives almost entirely in the free-autonomy zone.

8.2 The email send gate

Task: the agent drafts and sends email on your behalf — outreach, follow-ups, customer replies.

Permission analysis: drafting is perfectly reversible and should be ungated and unlimited — let the agent draft aggressively, badly, repeatedly; drafts are free. *Sending* is irreversible with reputational consequence: the wrong tone to a customer, a confidential detail to an outsider, a reply-all that should never have been. Medium-to-high consequence, irreversible: **gate the send.**

So you split the action, exactly as Section 5 prescribed: the agent holds `draft_email` freely and `queue_for_send` gated. The human sees the complete message — recipient, subject, body — and approves in one glance. Notice what the gate placement respects: the *consequence boundary* sits between draft and send, so that is where the gate goes. Not at "start working" (too early — you'd be approving intentions, which are unreadable), not after send (too late — that's an incident report, not a gate). And the design has a built-in maturity path: as the agent's track record accumulates *in your logs*, you can carve out an ungated fast lane — routine replies, to existing threads, from approved templates — while novel recipients and sensitive topics stay gated. The gate doesn't disappear as trust grows; it *retreats to the perimeter where it's still earning its attention cost.* That is what earned autonomy looks like mechanically, as opposed to rhetorically.

8.3 The payment gate

Task: the agent processes invoices — matches them to purchase orders, schedules payment, pays.

Permission analysis: matching and scheduling are reads and reversible writes — free. The payment itself is irreversible (on most rails) and consequential in proportion to amount. This is the case where every layer of the paper stacks into one design:

- **Schema as permission:** the payment tool's input schema caps `amount`; no confusion, error, or manipulation can exceed the cap, because the harness rejects the call before any model judgment is involved.
- **Scoped permission:** payees restricted to a pre-approved vendor list. Paying a new vendor isn't a bigger payment — it's a different, more dangerous action (this is precisely how invoice fraud works on human accounts-payable teams), so it gets its own, stricter gate.
- **Threshold gating:** under €200 to an approved vendor with a matched purchase order — auto-approve and log. Above that — human gate showing invoice, PO match, vendor history, and amount in one screen.
- **Restructured reversibility:** payments queue for a daily batch rather than executing instantly, creating a same-day cancellation window even below the gate threshold.

Observe that the finished design is not "agent + a yes/no button." It is graduated autonomy: full speed in the safe region, scoped speed in the middle, concentrated human judgment at exactly the two boundaries that matter (amount, new payee). The gate fires perhaps twice a day, so the human actually reads it. That is the shape competent agent systems converge on, and it is worth noticing that it is also the shape of a well-run finance function — because it is solving the same problem.

8.4 A founder's decision scenario

To make this operational: suppose you deploy an agent that handles inbound partnership inquiries — it reads the inquiry, researches the sender, drafts a response, and books intro calls. Where do the gates go?

Work the table. Reading and research: free. Drafting: free. Booking a calendar slot on *your* calendar: reversible with mild cost — autonomy with a scope (only designated slots, only 30 minutes) is defensible. Sending the reply: irreversible, and it's an outward-facing statement from your company to a potential partner — gate it, at least initially. Now the interesting question, the one the framework surfaces but cannot answer: after two hundred gated sends with a 98% unedited-approval rate, do you ungate routine replies? That's a judgment about your reputation's tail risk versus your attention budget — a genuine executive decision. The framework's contribution is that you are now making *that* decision, with data, instead of the unexaminable one ("do I trust AI?") you'd have been stuck with otherwise.

9. The Main Failure Mode: The Bored Human

If this system fails in practice, it will almost certainly not fail because a model went rogue past a well-built gate. It will fail because **the gate was technically present and humanly absent.**

The mechanism was previewed in Section 6 and deserves full weight here. Approval fatigue is not a discipline problem; it is a design outcome, and it follows a predictable arc. A gate fires often; nearly every proposal is fine; the approver's reading time per proposal decays from thirty seconds to three to zero; approval becomes a motor action. The system now has the *cost* of a gate (latency, human time) with the *safety* of no gate — the worst point in the entire design space. And the logs look wonderful right up until the incident, because "100% of proposals approved" is indistinguishable from "100% of proposals rubber-stamped."

Countermeasures, in order of leverage:

1. **Cut gate volume.** Every gate you remove from a routine action buys back attention for the gates that remain. If an approver handles more than a handful of gate decisions a day, redesign — usually by widening the auto-approve region for actions with a clean track record and tightening scope elsewhere to compensate.
2. **Watch decision time, not approval rate.** Median seconds-to-approve is your early-warning instrument. When it collapses, your gate has already died; you just haven't had the incident yet.
3. **Audit the flow-through.** Approved actions should still be sampled after the fact. A gate plus a periodic audit of what went through it is a control *system*; a gate alone is a control *moment*, and moments fail quietly.

A second failure mode is subtler: **the gate placed at the wrong boundary.** Gating "start the workflow" feels prudent but approves an intention rather than an action — you learn nothing decidable, and everything downstream runs ungated on the strength of that vague yes. The gate belongs at the last reversible moment before the consequence: send, pay, publish, delete. Everything before that boundary should be free precisely *because* the boundary holds.

And a third, for honesty's sake: permissions and gates govern *whether* an action may happen. They say nothing about whether an approved action then executes *correctly* — once, completely, and provably. An approved payment that silently fails, or silently executes twice, has passed every control in this paper and still burned you. That is deliberately out of scope today, and it is exactly where the next lesson picks up.

10. Why This Is an Organizational Idea Wearing Technical Clothes

A closing observation before the apparatus, because it's the reason this lesson sits in an operator's course rather than a security elective.

Everything in this paper — least privilege, scoped authority, graduated approval, audit of what flowed through — is a governance pattern that predates computers. Signing authority, spending limits, maker-checker controls, the separation of the person who requests a payment from the person who releases it: companies developed these because delegation at scale requires trust to be *structured*, not merely felt. What is genuinely new with agents is not the pattern but the medium: for the first time, the entire control structure can be *software* — enforced deterministically, logged completely, versioned like code, and adjusted in an afternoon instead of a reorg.

This is the deeper reason permission design belongs in the founder's own head rather than delegated wholesale to engineers. When agents do real work inside a company, the permission-and-gate map *is* the org chart's operational core — it encodes who (human or agent) may commit the company to what, and it becomes precise in a way paper policies never were. An orchestration layer over a knowledge substrate — an operator system in the sense this course has been building toward — is ultimately in the business of holding those keyrings and standing at those gates. The founders who can read and design that map will govern their AI-native companies deliberately. The ones who can't will discover their de facto permission policy the way most companies discover their de facto security policy: in the incident postmortem.

The next lesson completes the unit: given that an action has been permitted and approved, how do you make sure it executes reliably — once and only once, with a receipt that proves it? Permissions decide *may it happen*; reliability engineering decides *did it happen, exactly once*. You need both before an agent touches anything that matters.

Vocabulary

Permission — What an agent is mechanically able to request through its tools, enforced by the harness and its credentials rather than by prompt instructions. Includes scope: not just *which* tool, but with what constraints on targets, amounts, and inputs.

Least privilege — The principle of granting each agent (or each task) the minimum set of permissions its job requires. Shrinks the blast radius of any failure and, as a side effect, improves agent focus and evaluability.

Approval gate — A structural checkpoint in the harness where a proposed tool call is held until a human (or stricter policy) explicitly consents. Converts an irreversible action into a reversible proposal until the moment of approval.

Reversibility — The degree to which an action's real-world effect can be undone, and at what cost. The most useful single axis for pricing an action; classify the effect, not the API verb.

Blast radius — The full extent of harm possible if a given agent fails or is manipulated; bounded above by the union of its permissions, which is why revoking a permission is stronger than any instruction.

Approval fatigue — The decay of a frequently-fired gate into reflexive rubber-stamping. The dominant real-world failure mode of gated systems; countered by gate scarcity, decision-time monitoring, and post-hoc sampling.

Scope — The constraints attached to a permission: allowlisted recipients, amount caps, schema limits, template restrictions. Where most real permission design happens.

Harness — (From Tool Use I.) The deterministic software layer that receives the model's tool-call proposals and decides whether and how to execute them. The locus of every enforcement mechanism in this paper.

Reading Guide

Anthropic — Tool use overview (https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview). Read this with today's lens rather than last lesson's. On first pass (Tool Use I), the point was the loop: model proposes a call, your code executes, results return. On this pass, read for control surfaces. Notice that *you* define every tool — its name, its description, and its input schema — and that the model can only ever fill in the blanks you drew. Ask, at each step of the documented flow: where in this loop would a permission check live? (Answer: in your code, between receiving the tool-call proposal and executing it — the gap the documentation leaves as an exercise is exactly where this entire paper resides.) Notice also that nothing in the API enforces anything: the provider hands you the proposal mechanism; the permission and approval architecture is entirely your code and your responsibility. Fifteen minutes; the goal is to see the mechanical hooks that today's concepts hang on, not to memorize the schema format.

Walking Questions

1. Take one workflow you'd actually delegate to an agent this quarter. Walk each action it performs through the table in Section 7: which cell does it land in, and where is the single consequence boundary that deserves a gate?
2. The email case ungated drafting entirely and gated only the send. What is the equivalent split — the reversible/irreversible seam — in a payments workflow? In publishing content? In modifying a production database?
3. You review an agent system and find eleven approval gates. Using approval fatigue as the argument, make the case that this system is *less* safe than one with two gates. Where does the argument stop working?

4. Which permission in your own future stack belongs in the bottom-right cell — the one where the right answer is not a gate but "the agent doesn't hold this key at all"? What would have to be true, and demonstrated how, before you'd change that answer?

One-Sentence Takeaway

Give an agent the smallest keyring its job requires, let everything reversible run at full speed, and place a human gate at exactly the few irreversible boundaries where a bored click would cost you — because the safety of an agent system is decided by the consequence structure you build around it, not by the intelligence inside it.